Digital Marketing Campaign Selection

A 0/1 Knapsack problem solved with Dynamic Programming

Design and Analysis of Algorithms | Assignment 2026 | Batch 17

Team members

Member 1 (roll no.) | Member 2 (roll no.) | Member 3 (roll no.) | Member 4 (roll no.)

1. The Problem

- Total marketing budget: Rs 9 lakh. Six advertising campaigns.
- Each campaign has a cost and expected conversions; run it fully or not at all (0/1).
- Goal: choose the set of campaigns with the maximum total conversions within the budget.

Campaign	Cost (Rs lakh)	Conversions
C1	2	30
C2	3	45
C3	4	50
C4	5	70
C5	1	12
C6	6	80

64 possible subsets

Which one is best?

Answer: C2 + C4 + C5

cost 9 lakh, 127 conversions

2. Problem Modelling

- Input: costs c1..cn, conversions v1..vn, budget B = 9.
- Decision variable: xi = 1 if campaign i is selected, else 0.
- Objective: maximise Z = sum(vi * xi).
- Constraint: sum(ci * xi) <= B = 9.
- Output: the selected set S and the maximum conversions Z.
- An optimisation problem: many selections are feasible, we want the best one (NP-hard in general).

3. Data Structure Analysis

Data structure	Purpose	Time	Space
Arrays costs[], values[]	Store campaign data	O(n)	O(n)
2-D DP table dp[n+1][B+1]	Best value for (i, w); allows traceback of selected campaigns	O(nB)	O(nB)
1-D DP array dp[B+1]	Same values, one row reused (w loops downwards)	O(nB)	O(B)
Memo hash map (considered)	Top-down, only reachable states	O(nB)	O(nB)

Choice

- 2-D table: needed to recover WHICH campaigns were chosen and to show every intermediate value.
- 1-D array: O(B) memory; measured ~1.7-2x faster than the 2-D table for n >= 50 (n = 1000: 420 ms vs 837 ms).

4. Why Dynamic Programming?

- Optimal substructure: best plan for (first i campaigns, budget w) = best of 'reject i' or 'select i + best plan for budget w - ci'.
- Overlapping subproblems: naive recursion makes 2^n calls, but only (n+1)(B+1) distinct states (i, w).
- Indivisible campaigns + small integer budget: a table indexed by budget is tiny (7 x 10 cells here).
- Guaranteed optimum in O(nB) time, unlike Greedy (no guarantee) or Backtracking / Branch and Bound (exponential worst case).

5. Comparison of the Five Techniques

Technique	Verdict	Reason / counter-example
Divide and Conquer	Rejected	Halves {C1,C2,C3} / {C4,C5,C6} cannot be solved independently: the optimum uses only C2 from the left and C4, C5 from the right. Merging needs every budget split = a DP table.
Greedy (ratio)	Rejected (implemented)	Picks C1, C2, C3 = 125 < 127. Also B=10, items (1,2) and (10,10): greedy 2 vs optimum 10.
Dynamic Programming	SELECTED	Exact, O(nB), polynomial for small integer budgets.
Backtracking	Rejected	Explores up to 2^n include/exclude leaves (2^300 for the new requirement); repeats subproblems.
Branch and Bound	Rejected	Prunes with a fractional bound but is exponential in the worst case (e.g. value = cost, bound never prunes).

6. Algorithm and Pseudocode

State: dp[i][w] = max conversions using campaigns 1..i with budget w

Base: dp[0][w] = 0, dp[i][0] = 0 Answer: dp[n][B]

dp[i][w] = max(dp[i-1][w], dp[i-1][w - ci] + vi) if ci <= w, else dp[i-1][w]

```
KNAPSACK-DP(c[1..n], v[1..n], B)
  dp[0][w] = 0 for all w                          // base case
  for i = 1 to n:
    for w = 0 to B:
      dp[i][w] = dp[i-1][w]                       // reject campaign i
      if c[i] <= w and dp[i-1][w-c[i]] + v[i] > dp[i][w]:
          dp[i][w] = dp[i-1][w-c[i]] + v[i]       // select campaign i
  w = B;  S = {}                                  // traceback
  for i = n down to 1:
    if dp[i][w] != dp[i-1][w]:  S = S + {i};  w = w - c[i]
  return dp[n][B], S
```

7. Step-by-Step Execution (1-D dp[w], w = 0..9)

After	0	1	2	3	4	5	6	7	8	9
initial	0	0	0	0	0	0	0	0	0	0
C1 (2, 30)	0	0	30	30	30	30	30	30	30	30
C2 (3, 45)	0	0	30	45	45	75	75	75	75	75
C3 (4, 50)	0	0	30	45	50	75	80	95	95	125
C4 (5, 70)	0	0	30	45	50	75	80	100	115	125
C5 (1, 12)	0	12	30	45	57	75	87	100	115	127
C6 (6, 80)	0	12	30	45	57	75	87	100	115	127

Traceback from dp[6][9] = 127:

C6 skip (127 = 127) -> C5 TAKE (127 vs 125), w = 8 -> C4 TAKE (115 vs 95), w = 3
-> C3 skip (45 = 45) -> C2 TAKE (45 vs 30), w = 0 -> C1 skip

Selected: C2, C4, C5 **Cost: 3 + 5 + 1 = Rs 9 lakh** **Conversions: 45 + 70 + 12 = 127**

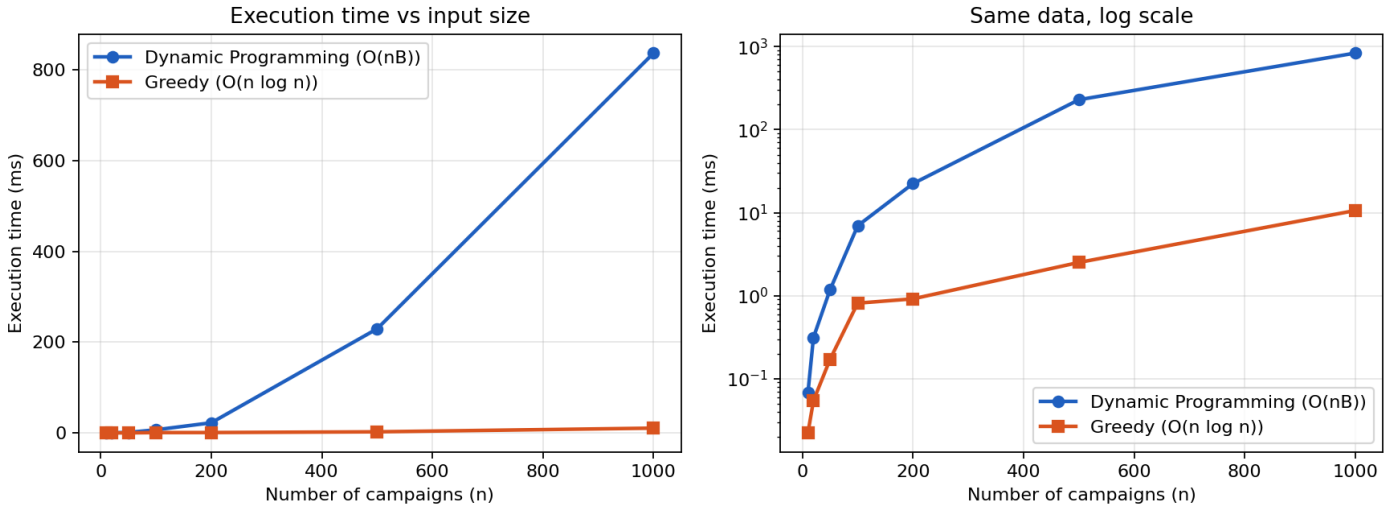
8. Complexity and Correctness

	Time	Space
2-D DP	Theta(nB)	Theta(nB) (70 cells here)
1-D DP	O(nB): sum(B - ci + 1) = 39 updates here	Theta(B) (10 cells here)
Best case	all ci > B: Theta(n + B)	-
Worst case	all ci = 1: exactly n*B updates	-
Greedy	O(n log n)	O(n)

Proof by induction on i (f(i, w) = true optimum for campaigns 1..i, budget w)

- Base: i = 0, only the empty set, so f(0, w) = 0 = dp[0][w].
- Step: an optimal set either excludes i (value f(i-1, w)) or includes i (value vi + f(i-1, w - ci), by cut-and-paste); the recurrence takes the max of both.
- End: dp[n][B] covers all subsets within budget, hence optimal. Verified against brute force on 300 random cases.

9. Greedy vs DP: Experimental Results



n	DP ms	Greedy ms	DP conv	Gr. conv
10	0.07	0.02	791	761
20	0.31	0.05	1209	1194
50	1.18	0.17	3298	3297
100	6.95	0.81	5120	5113
200	22.4	0.92	12529	12525
500	229.6	2.5	29439	29439
1000	837.3	10.6	59798	59798

- Same generated datasets for both algorithms; median of repeated runs (results/timing.csv).
- DP time grows ~quadratically (B grows with n); Greedy is ~80x faster at n = 1000.
- Greedy is not exact: it loses 3.8% at n = 10 and 2 conversions (1.57%) on the given instance; it can be arbitrarily bad in the worst case. DP is always optimal.

10. Changed Requirement: 300 campaigns, Rs 4,37,52,900

Plain DP at rupee precision	Value
Cell updates n x B	300 x 43,752,900 = 1.31 x 10^10
2-D table (4 bytes / cell)	about 52.5 GB
Time in Python (~14 M updates/s)	about 15 minutes per run

Improved approaches

- Cost scaling (unit Rs 1,000): round costs up, budget down -> 43,753 columns, always feasible, result >= OPT(B - k*unit): about 0.03% budget loss.
- Profit-scaling FPTAS: K = eps*vmax/n, DP over profit, O(n^3/eps) independent of B, value >= (1 - eps)*OPT.
- Synthetic 300-campaign test (LP bound 59,756): FPTAS (eps 0.1) 59,713 in 0.36 s; cost-scaled 59,713 in 0.02 s; Greedy 59,681.

11. Conclusion and References

- Dynamic Programming solves the 0/1 knapsack exactly: best plan = C2 + C4 + C5, Rs 9 lakh, 127 conversions.
- Greedy (ratio) gives 125 conversions: fast, O(n log n), but no optimality guarantee.
- DP costs O(nB): fine for 9 lakh, impractical for rupee-level budgets; scaling / FPTAS fixes this.

References

arXiv:2502.02920, Adaptive Budget Optimization for Multichannel Advertising Using Combinatorial Bandits (DP on a Multi-Choice Knapsack, Sec. 4.2).

Media Mix Optimization - Applying a Quadratic Knapsack Model, SCITEPRESS 2014 (MARMIND).

Kellerer, Pferschy, Pisinger. Knapsack Problems. Springer, 2004.

Ibarra and Kim. Fast approximation algorithms for the knapsack and sum of subset problems. J. ACM 22(4), 1975.

Code, report and results: Batches/Batch_17/ | Thank you